Labour Management System

Feature Requirements Document — V1

1. Product Objective

The Labour Management System is a simple digital **labour attendance and payment book for farmers**.

The farmer should be able to:

1. Mark daily attendance of individual labourers and labour teams.
2. Record what work was performed.
3. Track daily wages.
4. Manage labour and team information.
5. Assign contractual work and record agreed amounts.
6. Record money borrowed/advanced to labourers or teams.
7. Automatically calculate earnings, deductions, and outstanding balances.
8. View and download a complete statement for any labourer or team.

The application should be designed around the way a farmer actually works rather than around conventional HR/payroll software.

---

2. Core Concepts

The system has four major types of financial/work records:

A. Daily Wage Attendance

A labourer or team works on a particular day and is paid according to the daily wage.

Example:

> Deepak worked today as a daily wage labourer.
> Daily wage = ₹400
> Work = Crop spraying

System records:

> Earning = ₹400

B. Contract Work

A labourer or team is assigned a particular job for a fixed agreed amount.

Example:

> Deepak assigned grape pruning work for ₹8,000.

System records:

> Contract amount = ₹8,000
> Contract assigned date = today's date

The system does not calculate the amount based on hours worked.

C. Advance / Borrowed Money

A labourer or team receives money from the farmer.

Example:

> Deepak borrowed ₹3,000 for household groceries.

System records:

> Advance = ₹3,000

This amount increases the outstanding balance owed by the labourer.

D. Payments

When the farmer pays a labourer/team, the payment should eventually reduce the outstanding balance.

Although payment recording is not the main workflow described yet, the data model should be designed so that a **Payment** feature can be added without restructuring the application.

---

3. Application Navigation

The application should have the following main sections:

1. **Home / Dashboard**
2. **Today's Attendance**
3. **Labour**
4. **Labour Teams**
5. **Contract Work**
6. **Money / Advances**
7. **Statements**
8. **Settings**

The most important section is **Today's Attendance**.

---

4. Login

First Visit

When the farmer opens the application for the first time:

> Login

After successful login, the farmer should normally remain logged in.

The application should not make the farmer repeatedly log in every day.

---

5. Daily Attendance Gate

Attendance is the most important daily operation.

Whenever the farmer opens the application, the system checks:

> Has today's attendance been completed?

If **NO**, the application should prominently show:

> **Mark Today's Attendance**

The farmer should be able to enter the attendance workflow immediately.

Attendance Blocking Rule

If today's attendance has not been completed, the application should show an attendance prompt/ modal.

The farmer should not be able to use financial/statement features normally until today's attendance is completed.

Example:

Farmer opens app at 8 PM.

The system shows:

> **Today's attendance has not been completed.**
> Please mark today's labour attendance before continuing.

Buttons:

Mark Attendance

The farmer completes attendance.

After completion, the rest of the application becomes accessible.

Important

The system should **not force attendance immediately after midnight** if the farmer has not yet worked.

The application should provide a simple way to indicate:

No labour worked today

This should mark the attendance day as completed without requiring labour entries.

---

6. Today's Attendance

The attendance page should be designed for fast evening data entry.

The farmer should not have to navigate through many screens.

The page should show:

Date

Default:

Today's date

Attendance Categories

Two primary options:

1. Individual Labour

2. Labour Team

The farmer can add multiple entries under both categories.

Example:

Today's Attendance

Individual Labour:

- Deepak — Present
- Ramesh — Present
- Suresh — Present

Teams:

- Shinde Team — Present, 12 workers
- Pawar Team — Present, 8 workers

---

7. Individual Labour Attendance

When the farmer selects:

> **Add Individual Labour**

The system displays:

Labour

Dropdown/search:

> Select Labour

Only labourers already registered in the Labour section should appear.

Example:

> Deepak

Attendance Status

Options:

- Present
- Absent

Working Time

Optional:

> From: 8:00 AM
> To: 5:00 PM

This allows the farmer to record actual working hours.

The labour's normal/default timing should already be stored in their profile.

Actual attendance timing can override the default for that particular day.

Work Type

Required:

- Daily Wage
- Contract

However, if a contract already exists for the labourer, the contract should ideally be selected from the existing contract record rather than manually recreating it.

Work / Remark

Example:

> Crop spraying

or:

> Removed weeds from Plot 2

This becomes the description shown later in the labour statement.

Save

Example record:

> Deepak
> Present
> 8:00 AM – 5:00 PM
> Crop spraying
> Daily Wage

The system then automatically calculates the day's earning.

---

8. Individual Daily Wage Calculation

If Deepak's daily wage is:

> ₹400

And he is marked:

Present

Then:

Daily earning = ₹400

If he is marked:

Absent

Then:

Daily earning = ₹0

The system should not allow accidental earning for an absent labourer.

---

9. Individual Labour Early Exit / Partial Work

The first version should allow actual working time to be recorded but should **not automatically prorate the wage based on hours**.

Example:

Deepak's daily wage:

₹400

He works:

8 AM – 2 PM

Unless the farmer explicitly changes the earning/wage rule later, the system still records:

Daily earning = ₹400

The actual time is stored as attendance information.

This keeps V1 simple and avoids assumptions about how the farmer calculates half-day wages.

A future version can support:

Full Day / Half Day / Custom Wage

---

10. Labour Team Attendance

When the farmer selects:

Add Labour Team

The system asks for:

Team

Select existing team.

Example:

Shinde Team

Number of Labourers Present

Example:

12

This is important because the team size may change from day to day.

Example:

Team has 15 registered workers.

Today:

12 present

Tomorrow:

14 present

The system must calculate earnings based on the actual number present that day.

Working Time

Example:

8:00 AM – 5:00 PM

Optional.

Work / Remark

Example:

Removed weeds from Plot 3

Work Type

- Daily Wage
- Contract

---

11. Team Daily Wage Calculation

Suppose:

Team:

Shinde Team

Daily wage per labour:

₹300

Workers present:

12

Therefore:

Labour wage = 12 × ₹300 = ₹3,600

Additional team charges:

Car rent:

₹1,000

Manager fee:

₹100

Therefore:

Total daily team earning = ₹3,600 + ₹1,000 + ₹100

Total = ₹4,700

The system should automatically create this financial record.

---

12. Team-Specific Charges

A team can have additional charges:

Car Rent

Example:

₹1,000/day

Manager Fee

Example:

₹100/day

These values should be stored in the team profile as defaults.

However, the farmer should be able to modify them for a particular attendance day.

Example:

Normal car rent:

₹1,000

Today's car rent:

₹800

The attendance record should use:

₹800

instead of changing the permanent team profile.

---

13. Labour Management

The **Labour** section is the master database of individual labourers.

The farmer can:

- Add labour

- Edit labour
- View labour
- Deactivate labour
- View their statement

Individual Labour Information

Each labour record should contain:

Basic Information

- Name
- Hometown
- Aadhaar Card / Aadhaar reference
- Phone number — optional for V1

Wage Information

- Daily Wage
- Normal Working Time

Example:

Deepak

Daily Wage:

₹400/day

Normal Time:

8:00 AM – 5:00 PM

Hometown:

Junnar

---

14. Labour Teams

The **Labour Teams** section stores information about labour groups.

Each team should contain:

Team Information

- Team Name
- Manager Name

- Hometown

Wage Information

- Daily Wage Per Labour

Timing

- Normal Start Time
- Normal End Time

Additional Charges

- Car Rent
- Manager Fee

Example:

Shinde Team

Manager:

Ganesh Shinde

Daily Wage:

₹300/labour

Timing:

8 AM – 5 PM

Car Rent:

₹1,000/day

Manager Fee:

₹100/day

---

15. Contract Work

Contract work is separate from daily attendance.

The farmer can assign a fixed-price job to either:

- Individual Labour
- Labour Team

Create Contract

Fields:

Labour Type

- Individual
- Team

Labour / Team

Select from existing records.

Work

Example:

> Grape pruning - Plot 2

Finalised Amount

Example:

> ₹15,000

Contract Date

Automatically:

> Today's date

The contract date represents the day the work was assigned.

---

16. Contract Calculation

Contract work should not depend on:

- Number of hours
- Daily wage
- Number of workers

The agreed amount is the financial value of the contract.

Example:

> Shinde Team
> Work: Grape pruning

Contract amount: ₹25,000
Assigned: 15 August 2026

Statement should show:

15 Aug 2026 — Grape pruning — Contract ₹25,000

Whether payment is immediately made or remains outstanding should be handled through the payment/settlement system.

---

17. Money / Advances

The farmer should be able to record money given to a labourer or team.

Example:

Deepak borrowed ₹3,000 for groceries.

Add Money Entry

Fields:

Labour Type

- Individual
- Team

Labour / Team

Example:

Deepak

Amount

₹3,000

Date

Default:

Today's date

Reason / Remark

Example:

Household groceries

---

18. Advance Calculation

An advance should increase the outstanding amount owed by the labourer.

Example:

Deepak earns:

₹400

Deepak borrows:

₹100

Net outstanding for the day:

₹400 - ₹100 = ₹300

The system should retain both original transactions:

Earning

+₹400

Advance

-₹100

Net outstanding

₹300

The system should never overwrite the original earning.

---

19. Labour Statement

Every labourer should have a complete financial statement.

Example:

Deepak

Date	Work	Earning	Advance	Net	Outstanding
15 Aug	Crop spraying	₹400	₹100	₹300	₹300
16 Aug	Weeding	₹400	₹0	₹400	₹700
17 Aug	Harvesting	₹400	₹200	₹200	₹900

The statement should also display summary information.

Summary

Total Earned: ₹1,200

Total Borrowed: ₹300

Total Paid: ₹0

Outstanding: ₹900

---

20. Team Statement

Team statements should work similarly but account for multiple workers and team charges.

Example:

Shinde Team

15 August:

Workers:

12

Daily wage:

₹300

Labour earning:

12 × ₹300 = ₹3,600

Car rent:

₹1,000

Manager fee:

₹100

Total:

₹4,700

Statement:

Date	Workers	Work	Labour Wage	Car Rent	Manager Fee	Total
15 Aug	12	Weed removal	₹3,600	₹1,000	₹100	₹4,700

If the team borrowed:

₹2,000

Then:

Net outstanding = ₹4,700 - ₹2,000 = ₹2,700

---

21. Statement Requirements

The statement should contain:

Individual Labour

- Date
- Work
- Attendance status
- Working time
- Work type
- Daily earning
- Contract amount, if applicable
- Money borrowed
- Payment made
- Outstanding amount

Team

- Date
- Number of workers
- Work
- Daily wage per worker
- Labour wage total
- Car rent
- Manager fee

- Contract amount, if applicable
- Money borrowed
- Payment made
- Outstanding amount

---

22. Statement Summary

At the top of the statement:

Individual

Total Earned: ₹X
Total Contract: ₹X
Total Borrowed: ₹X
Total Paid: ₹X
Outstanding: ₹X

Team

Total Labour Earnings: ₹X
Total Car Rent: ₹X
Total Manager Fees: ₹X
Total Contract: ₹X
Total Borrowed: ₹X
Total Paid: ₹X
Outstanding: ₹X

---

23. Download Statement

The farmer should have:

Download Statement

The system should generate a clean statement that can be:

- Downloaded
- Printed
- Shared

The first version can generate a PDF.

---

24. Dashboard

The dashboard should be simple.

The farmer should immediately see:

Today's Status

Attendance: Completed / Pending

Today's Labour

Individual: 3
Teams: 2
Total workers: 27

Today's Estimated Earnings

₹X

Outstanding Labour Amount

₹X

Quick Actions

- Mark Today's Attendance
- Add Labour
- Add Team
- Add Contract
- Add Advance
- View Statements

The dashboard should prioritize **Today's Attendance** because that is the farmer's most frequent action.

---

25. Important User Flow

The primary real-world workflow should be:

```
Open App
   ↓
Login (only when required)
   ↓
Check Today's Attendance
   ↓
Attendance Pending?
   ↓
YES
   ↓
Mark Individual Labour
   +
Mark Labour Teams
```

```
    ↓
Enter Work / Time / Wage Type
    ↓
Complete Attendance
    ↓
Dashboard
    ↓
View Payments / Statements / Contracts / Advances
```

---

26. Example: Complete Day

Suppose today:

Individual Labour

Deepak:

> Present
> Crop spraying
> Daily wage
> ₹400

Ramesh:

> Present
> Weeding
> Daily wage
> ₹400

Suresh:

> Present
> Fertilizer application
> Daily wage
> ₹450

Individual total:

> ₹1,250

Labour Team

Shinde Team:

> 12 workers
> ₹300/worker

Weed removal
Car rent ₹1,000
Manager fee ₹100

Calculation:

12 × ₹300 = ₹3,600

Total:

₹3,600 + ₹1,000 + ₹100 = ₹4,700

Second Team:

8 workers
₹300/worker
Harvesting
Car rent ₹800
Manager fee ₹100

Calculation:

8 × ₹300 = ₹2,400

Total:

₹2,400 + ₹800 + ₹100 = ₹3,300

Today's Total Labour Cost

Individual:

₹1,250

Team 1:

₹4,700

Team 2:

₹3,300

Total:

₹9,250

Advance

Deepak borrowed:

₹100

Team 1 borrowed:

₹2,000

Total advances:

₹2,100

---

27. Financial Principle

The most important backend principle is:

Attendance and financial transactions must be stored separately.

The system should not simply store:

Deepak = ₹300

Instead, it should preserve the underlying events.

Example:

```
Attendance
    ↓
Deepak worked
    ↓
Daily earning = ₹400

Advance
    ↓
Deepak borrowed ₹100

Payment
    ↓
Farmer paid ₹200
```

Then the system calculates the current balance from these transactions.

This makes the system auditable and prevents the farmer's historical records from being accidentally destroyed.

28. Recommended Transaction Logic

For an individual:

```
Outstanding Balance
=
Previous Outstanding
+ Earnings
+ Contract Earnings
- Payments
- Advances
```

Where:

- Earnings = money earned through daily work
- Contract Earnings = assigned/finalised contract amount when treated as payable
- Advances = money already received by the labourer
- Payments = money paid/settled by the farmer

For a team:

```
Outstanding Balance
=
Previous Outstanding
+ Labour Wages
+ Car Rent
+ Manager Fee
+ Contract Amount
- Payments
- Advances
```

---

29. Important Distinction: Advance vs Payment

The system must distinguish between:

Advance / Borrowed

Money given to the labourer before final settlement.

Example:

> Deepak borrowed ₹1,000.

This increases the amount already received by Deepak.

Payment

Money paid against the outstanding labour balance.

Example:

> Farmer paid Deepak ₹5,000.

This reduces outstanding balance.

These should be two different transaction types in the backend.

---

30. Data Entities

The initial backend should conceptually contain these entities:

User

The farmer using the application.

Labour

Individual labour information.

LabourTeam

Information about a labour team.

Attendance

Daily attendance record.

AttendanceWorker / TeamAttendance

Detailed attendance information for the individual or team.

Contract

Contractual work assignment.

Advance

Money borrowed by labour/team.

Payment

Money paid/settled to labour/team.

Statement

The statement should preferably be **calculated from the underlying records**, rather than being stored as a separate financial truth.

---

31. Important Business Rules

Rule 1 — One attendance completion per day

The farmer should be able to edit today's attendance if they made a mistake.

The system should not create duplicate attendance days accidentally.

Rule 2 — Absent labour earns ₹0

Absent records should remain available for historical attendance but should not generate earnings.

Rule 3 — Team worker count can change

A team does not always have the same number of workers.

The attendance record must store:

> workers present today

rather than relying only on the team's default size.

Rule 4 — Default values vs actual values

Labour/team profiles contain defaults.

Daily attendance contains actual values.

Example:

Team default:

> Car rent ₹1,000

Today's actual:

> ₹800

The attendance record should use ₹800 without changing the default ₹1,000.

Rule 5 — Historical records should not change automatically

If Deepak's wage changes from:

₹400 → ₹450

future attendance should use ₹450.

Previous attendance should continue showing:

₹400

Rule 6 — Deleting labour should not delete history

If Deepak stops working:

Deactivate Deepak

Do not delete his historical attendance, advances, contracts or statements.

---

32. V1 Scope

The first version should focus on these features:

Must Have

- Login
- Dashboard
- Today's attendance
- Individual labour attendance
- Team attendance
- Labour management
- Team management
- Daily wage calculation
- Contract assignment
- Advance/borrowed money
- Individual statement
- Team statement
- Outstanding calculation
- PDF statement download

Not Required Initially

- WhatsApp integration
- SMS
- Notifications

- GPS tracking
- Biometric attendance
- Worker mobile app
- Payroll automation
- Aadhaar verification
- Online payments
- Multiple farms
- Complex permissions
- AI features

The goal of V1 is to replace the farmer's **physical labour notebook/khata** with a simple digital version.

---

33. Design Philosophy

The application should **not look like corporate HR software**.

The farmer should be able to open it in the evening and finish the day's records quickly.

The design should prioritize:

- Large buttons
- Simple language
- Minimal typing
- Searchable labour names
- Dropdowns
- Default values
- Clear ₹ amounts
- Today's date automatically selected
- Fast attendance entry
- Mobile-first interface

The primary question for every screen should be:

> **Can a farmer understand this without someone explaining the software to them?**

---

34. Core Product Loop

The entire application revolves around this loop:

```
LABOUR MASTER DATA
        ↓
WHO WORKED TODAY?
        ↓
ATTENDANCE
        ↓
WHAT DID THEY DO?
```

```
        ↓
EARNING CALCULATION
        ↓
ADVANCE / PAYMENT
        ↓
OUTSTANDING BALANCE
        ↓
LABOUR STATEMENT
```

Contract work operates alongside this:

```
LABOUR / TEAM
        ↓
CONTRACT ASSIGNED
        ↓
WORK + AGREED AMOUNT
        ↓
STATEMENT
        ↓
PAYMENT / OUTSTANDING
```

This should be the fundamental architecture of the product.